01 / THE ARGUMENT

Closest pair, revisited.

A corrected companion to my first lecture at AWaDH. The original notes and Python script remain available as a record of that lecture.

The question, precisely

Given at least two points in the Euclidean plane, return one pair with minimum squared distance. Repeated coordinates are distinct observations and give a distance of zero. This implementation accepts integer coordinates so that distance comparisons remain exact.

Sort once. Preserve the order.

Sort the points by x and separately by y. Split the x order at its midpoint. Partition the y order into those same two halves with a single stable pass. An x-rank identifies membership even when many points share an x coordinate.

The only remaining candidates

After solving both halves, let d be the smaller distance found. A better cross-boundary pair must lie within d of the divider. Keep that strip in y order. For each point, compare at most the next seven points, stopping sooner when the vertical gap alone is already too large.

Why seven is enough

Look above a strip point through height d. The relevant 2d-by-d rectangle has two halves, one for each recursive side. Subdivide each half into four d/2-by-d/2 cells. Two same-side points in one cell would be closer than d, contradicting the recursive result. At most eight points, including the starting point, can occupy this rectangle. Duplicate coordinates are handled before recursion.

What the bound actually says

Sorting costs O(n log n). Each recursive level partitions and scans in O(n), giving $T(n) = 2T(n/2) + O(n)$ and O(n log n) overall. This counts comparisons and arithmetic operations; very large integers add bit-arithmetic costs. Live auxiliary storage is O(n), with O(log n) recursive depth.

02 / FROM THE ORIGINAL TO THE REVISION

Small details. Real consequences.

The changes below concern the closest-pair argument and implementation; the archived files themselves are unchanged.

A consistent boundary contract

The original brute-force helper returns different tuple shapes for fewer than two points. The revision rejects such input with ValueError. Every successful call returns indices, point coordinates, and an exact squared distance.

Identity is not the same as position

Testing whether a coordinate tuple appears in the left list is linear work per point and can send duplicates into the wrong partition. Stable x-ranks make the y partition linear and unambiguous. Adjacent equal coordinates in x order return a zero-distance answer immediately.

A strip bound matched to its proof

The lecture mentions six following points; the original Python loop checks four. This companion uses the standard conservative bound of seven and a vertical-gap early exit. The proof and the loop now describe the same algorithm.

One answer, even when several tie

The revision intentionally returns one minimum pair. It does not enumerate every equally close pair. That distinction matters: n identical points have n(n-1)/2 zero-distance pairs, so listing all of them cannot have an O(n log n) output bound.

A correction to the conclusion

On page 6 of the original PDF, the final time bound should be O(n log n), not logarithmic time. Re-sorting the strip at every recursive call would also lose this bound; the revision preserves the y order instead.

A background clarification: a space is compact when the whole space has the finite-subcover property. This does not require every subset to be compact, as the wording on page 1 suggests.

03 / TRY IT, THEN CHALLENGE IT

Evidence you can rerun.

Python 3.10 or newer. No third-party packages are required by the implementation or its tests.

Keep the two Python files together

Download closest_pair.py and test_closest_pair.py into the same folder. From that folder, run:

```
python closest_pair.py
python -m unittest -v test_closest_pair.py
```

A small example

For [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)], the closest points are (2, 3) and (3, 4), at input indices 0 and 5. The returned squared distance is 2. Take its square root only if an approximate Euclidean distance is needed.

Checked on 14 October 2026

All 12 test methods pass. They include 1,000 reproducible random inputs compared with an independent all-pairs oracle and all 502 subsets of a 3-by-3 integer grid containing at least two points. The assertions also check that the reported points and indices agree.

Additional cases cover duplicates, horizontal and vertical lines, equal x coordinates, cross-boundary pairs, tied distances, very large integers, invalid input, generators, and preservation of the original input.

What the tests do not establish

Passing these cases supports correctness; it does not prove the time bound or benchmark production performance. The complexity argument is on page 1. Floating-point coordinates are deliberately rejected. Accepting them would need an explicit policy for roundoff, overflow, and near-equal distances.

Further reading and source

Princeton Algorithms, ClosestPair reference
Princeton Algorithms, implementation and strip argument
My first lecture: original materials, companion, and Python downloads